

Classification in Health Insurance Cost

YAKIRA MCKAY (759)
YMCKAY3@GATECH.EDU
4/19/2026

Abstract:

This study investigates how machine learning models can be used to classify Medicare beneficiaries into healthcare cost tiers using the CMS DE-SynPUF dataset. A key objective is to examine the trade-off between predictive accuracy and interpretability, as well as to assess whether demographic variables disproportionately influence model predictions. The response variable is constructed from total reimbursement amounts and transformed to address skewness in cost data.

Five models are evaluated: logistic regression, decision tree, random forest, gradient boosting, and k-nearest neighbors (KNN). Logistic regression serves as the baseline model. Hyperparameter tuning is conducted on a validation set, and model performance is evaluated using recall, F1 score, and Cohen's kappa for the high-cost class, reflecting the importance of accurately identifying high-cost patients.

Results show that logistic regression achieves the strongest performance, while ensemble methods such as gradient boosting and random forrest follow behind. Feature importance analysis indicates that chronic disease variables are the primary drivers of classification, with limited influence from demographic variables.

Introduction:

Health insurance pricing is one of the most consequential applications of predictive modeling in the insurance industry. Insurers must accurately estimate the expected cost of claims for each policyholder to set premiums that are both competitive and financially sustainable. Mispricing can lead to significant consequences, including profitability issues, inefficient resource allocation, and poor customer outcomes such as overpricing low-risk patients and underpricing high-risk patients. As healthcare costs continue to rise, the ability to accurately identify high-cost patients has become increasingly valuable for insurers, healthcare administrators, and policymakers.

This analysis leverages data from the Centers for Medicare and Medicaid Services (CMS) Linkable 2008–2010 Medicare Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF), specifically the DE1.0 Sample 1 2009 Beneficiary Summary File. The primary objective is to build a binary classification model that predicts whether a Medicare beneficiary falls into a High Cost or Standard Cost category. Total cost is calculated by summing Medicare reimbursements across inpatient, outpatient, and carrier services. While cost distributions naturally suggest a three-tier structure (low, medium, and high), the distinction between low and medium cost beneficiaries has limited practical impact for pricing decisions. The more relevant business problem is identifying the top 33% of spenders—those most likely to strain resources—from the remaining 67%. Accordingly, the response variable is defined as a binary outcome using this 33/67 split.

A central question in this analysis is whether to prioritize predictive accuracy or interpretability—that is, the ability to explain which variables drive classification into the High Cost category. In practice, both are important: accuracy supports reliable pricing decisions, while interpretability enables transparency and justifiability.

To explore this trade-off, the analysis begins with logistic regression and decision trees for interpretability, then evaluates ensemble methods, including random forest and gradient boosting, to improve predictive performance. Ensemble methods combine multiple models rather than relying on a single classifier, typically increasing accuracy and reducing variance. K-nearest neighbors (KNN), a non-parametric, instance-based method, is also included to assess how a distance-based approach performs relative to model-based methods. The goal is to determine which modeling approach provides meaningful improvements and yields the most effective classification model.

Several data challenges are also considered. Medical cost data is typically heavily right-skewed, with a small proportion of patients accounting for a large share of total spending, making careful response construction essential. The dataset also includes a mix of continuous, binary, and categorical variables, requiring thoughtful preprocessing and encoding. Ethical considerations are critical as well. Variables such as race, sex, and geographic location can introduce bias if not handled carefully. In practice, models that rely heavily on these features may reinforce existing healthcare disparities rather than reflect true clinical risk. As a result, part of this analysis evaluates whether demographic variables disproportionately influence High Cost predictions in ways that raise fairness concerns.

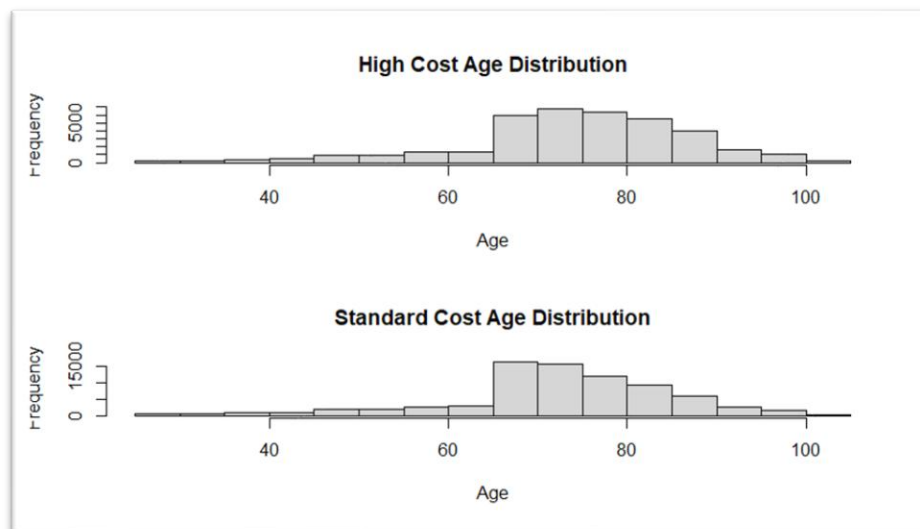
The remainder of the report is structured as follows: the data section covers preprocessing and variable selection, the methodology section outlines the models used, the results section compares performance and feature importance, and the discussion section interprets findings while addressing ethical implications and limitations.

Problem Statement/Data Sources

Our dataset is sourced from the Centers for Medicare and Medicaid Services (CMS) Linkable 2008–2010 Medicare Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF), published on January 15, 2013. Specifically, this analysis uses the DE1.0 Sample 1 2009 Beneficiary Summary File, which contains 114,538 synthetic Medicare beneficiary records across 32 variables. Each record represents a synthetic beneficiary and includes information on demographics, Medicare enrollment, health status, and reimbursement amounts. Because this dataset is synthetic and designed to preserve the structure of real Medicare claims data while protecting privacy, findings from this analysis are intended for methodological demonstration and cannot be directly generalized to real populations.

Variable 1 serves as the Beneficiary Code, a unique identifier for each individual. Variables 2–5 and 7–8 represent demographic information, including date of birth, date of death, sex, race, state code, and county code. Although these variables are numerically encoded, they are treated as categorical variables in this analysis.

Using date of birth, an age variable was derived, with values ranging from 26 to 101. The distribution is concentrated among older beneficiaries, with both the mean and median age equal to 73. This indicates a strong concentration of individuals above age 60, which is consistent with the Medicare population.



The racial distribution is highly imbalanced, with approximately 82% of observations classified as White, 10% as Black, 4% as Other, and 2% as Hispanic. This imbalance is important to consider when interpreting model results, particularly in the context of fairness and representation.

Variable 6 is the End-Stage Renal Disease (ESRD) indicator, a binary flag denoting whether a beneficiary has been diagnosed with ESRD, and is treated as a categorical variable.

Variables 9–12 represent the number of months a beneficiary was enrolled in Medicare Part A, Part B, Medicare HMO, and Part D coverage. These variables are treated as continuous and serve as indicators of coverage duration and plan participation.

Variables 13–23 capture chronic condition diagnoses and include binary indicators for eleven conditions: Alzheimer’s disease or related disorders, heart failure, chronic kidney disease, cancer, chronic obstructive pulmonary disease (COPD), depression, diabetes, ischemic heart disease, osteoporosis, rheumatoid arthritis or osteoarthritis, and stroke or transient ischemic attack. These variables are treated as binary categorical predictors.

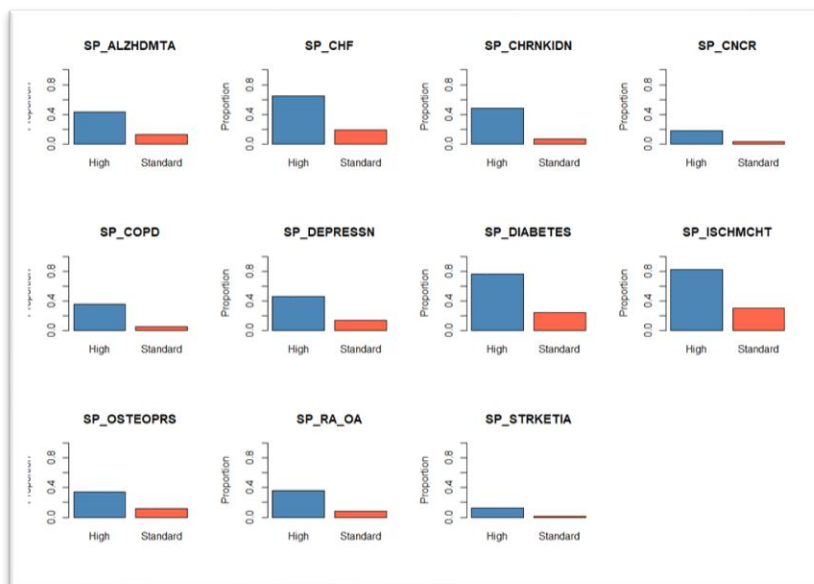


Figure 2: Chronic Disease Distribution

Across all chronic disease indicators, individuals with a condition show a higher proportion of High Cost classification compared to Standard Cost, suggesting a strong association between chronic disease and increased healthcare costs.

Variables 24–32 contain Medicare reimbursement amounts broken down across three service categories: inpatient (variables 24–26), outpatient (variables 27–29), and carrier (variables 30–32). Each category split into Medicare-paid, beneficiary responsibility, and primary payer amounts.

Proposed Methodology

Before beginning the analysis, the data is prepared through feature selection, variable transformation, and additional calculations. The Beneficiary Code (variable 1) is excluded, as it serves only as a unique identifier and provides no predictive value. Date of Death (variable 3) is also removed, along with any records where it is not null. Death is typically an outcome of underlying health conditions rather than a predictor, and including this variable would introduce data leakage into the model.

Categorical variables are recoded prior to modeling. Chronic condition indicators (variables 13–23) are originally encoded as 1 (yes) and 2 (no) and are recoded to 1 (yes) and 0 (no) for consistency and proper model interpretation. The End-Stage Renal Disease (ESRD) indicator (variable 6), encoded as Y (yes) and 0 (no), is similarly converted to 1 and 0. Sex (variable 4), originally encoded as 1 (male) and 2 (female), is also recoded to a binary 1/0 format. Race (variable 5) and State Code (variable 7) are one-hot encoded due to their multi-category nature.

State Code requires additional preprocessing. The codes are assigned alphabetically by state name, with the exception of four non-state categories: DC (09), Puerto Rico (40), Virgin Islands (48), and Other (54). To reduce dimensionality and improve interpretability, states are grouped into five regions: Northeast, South, Midwest, West, and Other, where “Other” includes the non-state codes. Additionally, Date of Birth (variable 2) is transformed into a derived age variable representing each beneficiary’s age in 2009, which is treated as continuous.

The response variable, Cost_Factor, is constructed using Medicare reimbursement amounts across three service categories: inpatient (variable 24), outpatient (variable 27), and carrier (variable 30). These variables are summed to produce the total primary payer cost per beneficiary. Because healthcare cost data is heavily right-skewed, a $\log(x + 1)$ transformation is applied to reduce skewness and limit the influence of extreme values. The transformed values are then used to define a binary outcome: beneficiaries in the top 33% of the distribution are classified as High Cost (1), while the remaining 67% are classified as Standard Cost (0).

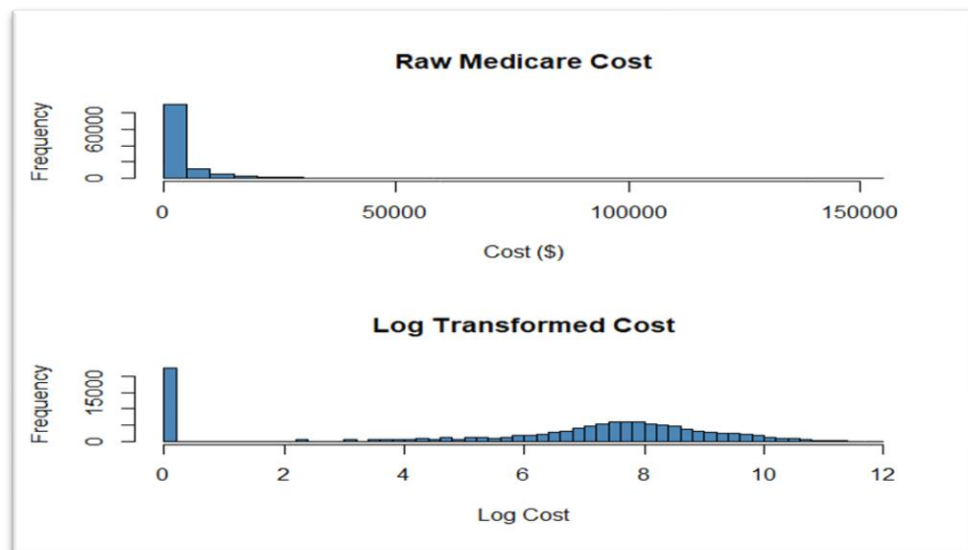


Figure 1: Raw Medicare Cost Distribution vs Log Transformation Distribution Cost

All other reimbursement variables, including those reflecting beneficiary responsibility or secondary payments, are excluded, as they are not directly relevant to Medicare’s primary reimbursement totals. The final dataset used for modeling contains 112,752 observations and 20 predictor variables.

The data will be partitioned into three sets: 70% for training, 15% for validation, and 15% for testing. A validation set is included to enable hyperparameter tuning on held-out data prior to final evaluation on the test set, helping prevent overfitting during model selection. Stratified sampling will be used across all three partitions to preserve the class distribution of Cost_Factor.

Five supervised binary classification models will be evaluated on the training data: logistic regression, decision tree, random forest, gradient boosting, and k-nearest neighbors (KNN).

Logistic regression estimates the probability of an observation belonging to a given class by applying the logistic function to a linear combination of predictors. These predicted probabilities are then converted into class labels using a specified classification threshold. This model serves as the baseline due to its simplicity and interpretability, providing a reference point for evaluating more complex models. The model will be implemented using `glm()` in R, and classification thresholds of 0.5, 0.6, 0.7, and 0.8 will be evaluated. Because of the factor level ordering used in this analysis, the model returns the probability that an observation belongs to the “Standard” class. Therefore, observations with predicted probabilities below the selected threshold will be classified as “High.” Key assumptions include independence of observations, linearity in the logit, absence of strong multicollinearity, and a sufficiently large sample size. These assumptions are considered during model fitting.

A decision tree is a nonparametric model that partitions the data through a series of binary splits, producing increasingly homogeneous subsets until terminal nodes assign class labels. The model will be built using `rpart()` in R. Hyperparameter tuning will focus on the complexity parameter (*cp*) and *minsplit*. The *cp* parameter controls the minimum improvement required for a split, with smaller values producing more complex trees. The *minsplit* parameter defines the minimum number of observations required to attempt a split, where larger values yield simpler models. The *minbucket* parameter is set by default to one-third of *minsplit* and represents the minimum number of observations allowed in a terminal node. The values explored for *cp* are 0.0001, 0.0005, 0.001, and 0.005, while *minsplit* values of 500 and 750 are tested, resulting in eight model configurations.

Random forest is a nonparametric ensemble method that extends decision trees by constructing multiple trees using bootstrap samples of the data and random subsets of predictors at each split. Predictions are aggregated through majority voting. This approach reduces variance and typically improves predictive performance. The model will be implemented using `randomForest()` in R. Hyperparameter tuning will explore *min.node.size*, which controls the minimum number of observations in a terminal node, and *mtry*, which specifies the number of predictors considered at each split. Values of 2, 6, and 12 will be tested for *min.node.size*, and values of 4 and 5 for *mtry*, based on the square root heuristic for the number of predictors (approximately $\sqrt{20} \approx 4.5$). This results in six model configurations.

Gradient boosting is an ensemble method that builds trees sequentially, where each new tree is trained to correct the residual errors of the previous model. This iterative approach often leads to strong predictive performance. The model will be implemented using `gbm()` in R with an initial maximum of 300 trees. The response variable is encoded as 0 and 1 to align with the Bernoulli distribution. Internal cross-validation is used to determine the optimal number of trees. Hyperparameter tuning will focus on *shrinkage* (learning rate) and *n.minobsinnode* (minimum observations per terminal node). Smaller shrinkage values slow learning but can improve generalization, while larger values increase training speed. The values explored are 0.1 and 0.05 for *shrinkage*, and 10, 20, and 30 for *n.minobsinnode*, resulting in six configurations. Due to computational cost and dataset size (~78,000 training observations), a relatively higher shrinkage and capped tree count are used.

K-nearest neighbors (KNN) is a nonparametric, instance-based method that assigns class labels based on the majority class among the *k* nearest observations. Because KNN relies on distance calculations, all numerical features are scaled and standardized to prevent variables with larger magnitudes from dominating. Categorical variables are appropriately encoded. The tuning parameter is *k*, the number of neighbors considered. The values tested are 3, 7, 11, 15, 21, and 31.

The primary evaluation metric is recall for the High Cost class. In this context, failing to identify a high-cost beneficiary (false negative) carries greater consequences than incorrectly classifying a standard-cost beneficiary as high-cost (false positive). Recall measures the proportion of true High Cost cases correctly identified, making it the most appropriate metric for this objective. Additional metrics include F1 score and Cohen's kappa. F1 balances recall and precision, addressing cases where high recall may come at the expense of excessive false positives. Cohen's kappa measures agreement between predicted and actual classifications while accounting for chance, providing a more robust assessment in the presence of class imbalance.

After evaluating models on the validation set, parameters from with the best results from each model tupe will be extracted and used to retrain the final models that will be evaluated on the test set. The final model is selected based on the highest recall for the High Cost class, subject to maintaining acceptable F1 score and Cohen's kappa to ensure balanced and reliable performance. Feature importance will be extracted to identify the variables that most strongly influence High Cost classification. Particular attention will be given to demographic variables such as race, sex, and geographic location to assess whether they disproportionately influence predictions. In practice, over-reliance on these variables may reflect systemic disparities rather than true clinical risk, raising important fairness considerations.

Analysis and Results

Hyperparameter Tuning Analysis

All five models were trained, tuned and evaluated using the validation set. Detailed results for each configuration are provided in the appendix.

For logistic regression, adjusting the classification threshold directly impacts the balance between recall and overall model performance. While increasing recall for the High Cost class is important, overemphasizing recall can lead to a large number of false positives, which may also have negative implications. Because the model outputs the probability of the "Standard" class, increasing the less than threshold results in more observations being classified as "High," which in turn increases recall. However, this comes at the expense of precision. As a result, greater emphasis was placed on the F1 score, which balances recall and precision. The results show that the F1 score is highest at a threshold of 0.7, suggesting that this threshold provides the best balance between correctly identifying High Cost individuals and limiting false positives.

For the decision tree, the complexity parameter (cp) and minsplit (which determines minbucket) were tuned. The cp parameter had a greater influence on performance compared to minsplit. A cp value of 0.001 produced the best results across both minsplit values (500 and 750), with minsplit = 750 yielding the highest recall. This aligns with expectations, as larger minsplit values reduce overfitting and improve generalization. While the F1 score ranked second among models, the kappa score ranked lower, suggesting that some of the model's performance may be influenced by chance.

For the random forest model, the best-performing configuration for recall used mtry = 4 and min.node.size = 12. This result is consistent with theory, as a smaller mtry increases model diversity and reduces overfitting, while a larger min.node.size promotes more stable and generalized trees. Together, these parameters provide a good balance between variance and bias.

For gradient boosting, tuning required balancing the learning rate (shrinkage) with the number of trees, while also considering computational cost. Larger shrinkage values required fewer trees to reach optimal performance,

whereas smaller shrinkage values required more iterations. The optimal configuration on the validation set was shrinkage = 0.05 and n.minobsinnode = 20. This combination reflects a slower learning rate with a moderate node size, which helps control overfitting while maintaining strong predictive performance. Both F1 score and kappa ranked second across models, reinforcing that the model maintains balanced performance alongside strong recall.

For K-nearest neighbors, the optimal value of k for maximizing recall was 21. This suggests that considering a larger neighborhood improves generalization by reducing sensitivity to noise. This result is supported by the F1 and kappa scores, which also ranked second relative to the other models, indicating a good balance between bias and variance.

Final Testing Evaluation

The optimal hyperparameter combination for each model was selected, and the models were retrained. Variable importance was extracted for logistic regression, decision tree, random forest, and gradient boosting. Since K-nearest neighbors (KNN) is a distance-based model, variable importance cannot be directly interpreted.

The top 10 variables for logistic regression, decision tree, and boosting were all related to chronic disease indicators. For random forest, the top 10 also included age (ranked 5th) and region (ranked 10th). Across all four models, chronic kidney disease and ischemic heart disease consistently appeared within the top three predictors. Diabetes was also ranked in the top three for three of the four models, with logistic regression as the exception, where cancer was ranked instead.

On the test set, logistic regression achieved the highest recall (0.829), followed by gradient boosting (0.708), random forest (0.696), decision tree (0.693), and KNN (0.687). Logistic regression also produced the highest F1 score (0.737) and a kappa value of 0.585. The remaining models exhibited similar performance, with F1 scores ranging from 0.711 to 0.727 and kappa values between 0.574 and 0.598.

The improved performance of boosting relative to random forest can be attributed to its sequential learning process, where each tree is built to correct the errors of the previous one, allowing it to better capture complex patterns in the data. Random forest, while also an ensemble method, builds trees independently using bagging, which reduces variance but does not iteratively refine predictions.

Both ensemble methods outperform the single decision tree, which is more prone to overfitting and lacks the stability gained from aggregating multiple models. Finally, KNN performs the weakest among the models, likely due to its reliance on distance-based calculations in a high-dimensional space, where feature scaling and the presence of many predictors can reduce its ability to effectively distinguish between classes.

McNemar's test was conducted to assess differences in classification outcomes between models. Pairwise comparisons between logistic regression and all other models resulted in chi-square statistics ranging from 615.7714 to 1688.2540, with corresponding p-values effectively equal to 0, indicating statistically significant differences in predictions. Similarly, comparisons among gradient boosting, random forest, decision tree, and KNN also yielded p-values near 0, indicating that differences in predictions across these models are statistically significant, despite relatively similar aggregate performance metrics.

Overall, while metric values across ensemble and non-parametric models are close, McNemar's test results indicate that the models produce significantly different classification decisions at the observation level.

Model	Recall	F1	Kappa
Logistic_Classification	0.829	0.737	0.585
Boost_Model	0.708	0.727	0.598
Random_Forest	0.696	0.722	0.593
Decision_Tree	0.693	0.711	0.574
KNN_Model	0.687	0.712	0.577

Table 1: Best Models by Validation Test

Conclusion:

In conclusion, Logistic regression performed the best when evaluating recall and F1, with a competitive Kappa score, indicating that it not only captured a large proportion of High Cost individuals but also maintained a strong balance between precision and recall. It is important to note that this performance is influenced by threshold tuning, which allows the model to prioritize recall while still maintaining reasonable precision. When comparing Ensemble methods to the single decision tree, Ensemble Methods produce better results.

A key finding across all models is that chronic disease indicators are the primary drivers of high-cost classification, with conditions such as chronic kidney disease, ischemic heart disease, and diabetes consistently ranking among the most important predictors. In contrast, demographic variables such as race, sex, and geographic region show limited influence. This suggests that the models are capturing clinical risk rather than relying heavily on demographic characteristics, helping to mitigate concerns around fairness and bias.

These findings reinforce that strong predictive performance can be achieved without over-reliance on sensitive demographic features. However, further analysis could strengthen this conclusion by retraining models with demographic variables removed and evaluating any resulting changes in performance. This would provide additional validation of whether these variables are truly unnecessary for accurate classification.

Future work could also explore dimensionality reduction techniques such as Principal Component Analysis (PCA). PCA may improve model efficiency and potentially enhance predictive performance by reducing noise and multicollinearity in the data. However, this would come at the greater cost of interpretability, as the resulting components are linear combinations of the original variables and are more difficult to explain in a business or healthcare context.

In conclusion, this analysis demonstrates that machine learning models can effectively identify high-cost Medicare beneficiaries using clinical indicators. While ensemble methods provide strong and consistent performance, logistic regression achieved the highest recall and F1 score when the classification threshold was tuned to prioritize high-cost identification. These findings highlight the importance of aligning model configuration with the objective function, as well as balancing performance, interpretability, and fairness in model development.

Lesson Learned

This analysis highlighted several practical considerations when applying machine learning models to large healthcare datasets. One key takeaway is the impact of computational constraints on model selection. While additional models such as support vector machines (SVM) were considered, however, given the size of the dataset, reducing the feature space to a computationally feasible level required retaining a relatively small number of

components, which explained only a limited proportion of the total variance. This reinforces that model choice is not solely driven by theoretical performance, but also by available computational resources and scalability.

Another important insight is that not all hyperparameters contribute equally to model performance. During tuning, certain parameters, such as the complexity parameter (*cp*) in decision trees, had a much larger impact on evaluation metrics compared to others like *minsplit*. This emphasizes the importance of understanding which parameters meaningfully influence model behavior, rather than treating all tuning parameters as equally important.

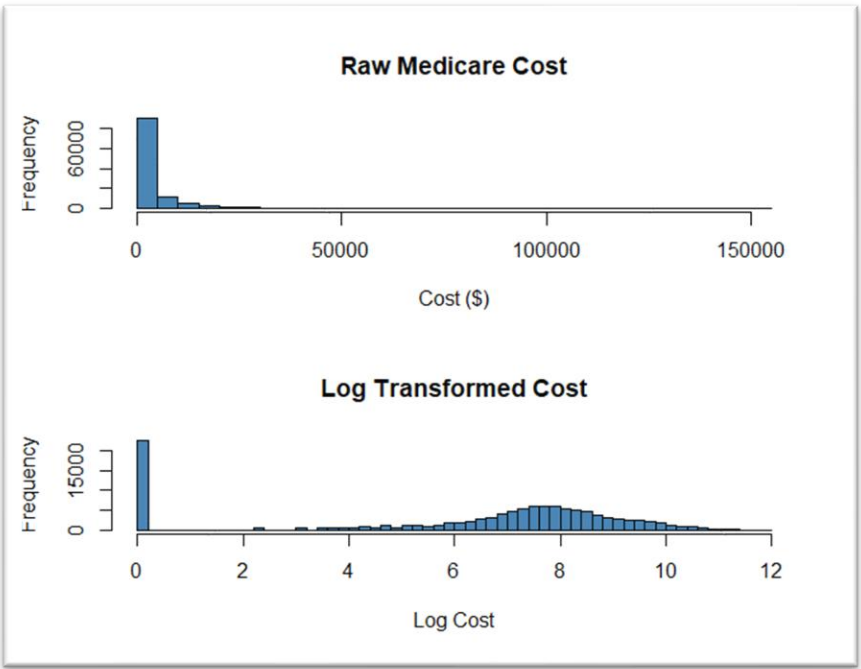
The analysis also reinforced the importance of aligning evaluation metrics with the problem context. Prioritizing recall for the High Cost class ensured that the models focused on identifying high-risk beneficiaries, which is more relevant for healthcare cost management than overall accuracy, however precision still needs to be considered to reduce the number of false positives and maintain balanced model performance.

Additionally, the results demonstrated that increased model complexity does not always lead to significantly better performance. Logistic regression, the simplest model evaluated, achieved the strongest results when the classification threshold was optimized. This highlights the importance of model configuration, as well as the trade-offs between simplicity, interpretability, and predictive performance.

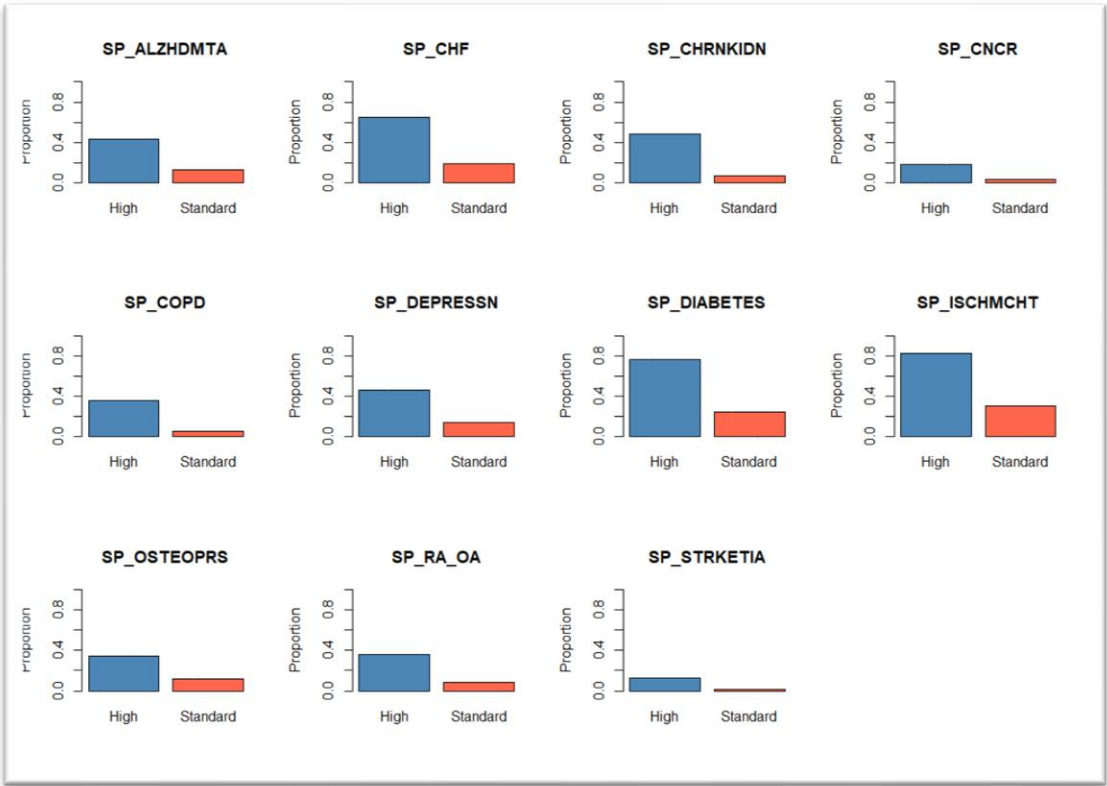
Finally, this project underscored the importance of thoughtful data preparation. Feature engineering decisions, such as transforming skewed cost data and properly encoding categorical variables, had a direct impact on model performance and stability.

Appendix – Charts and Visualizations

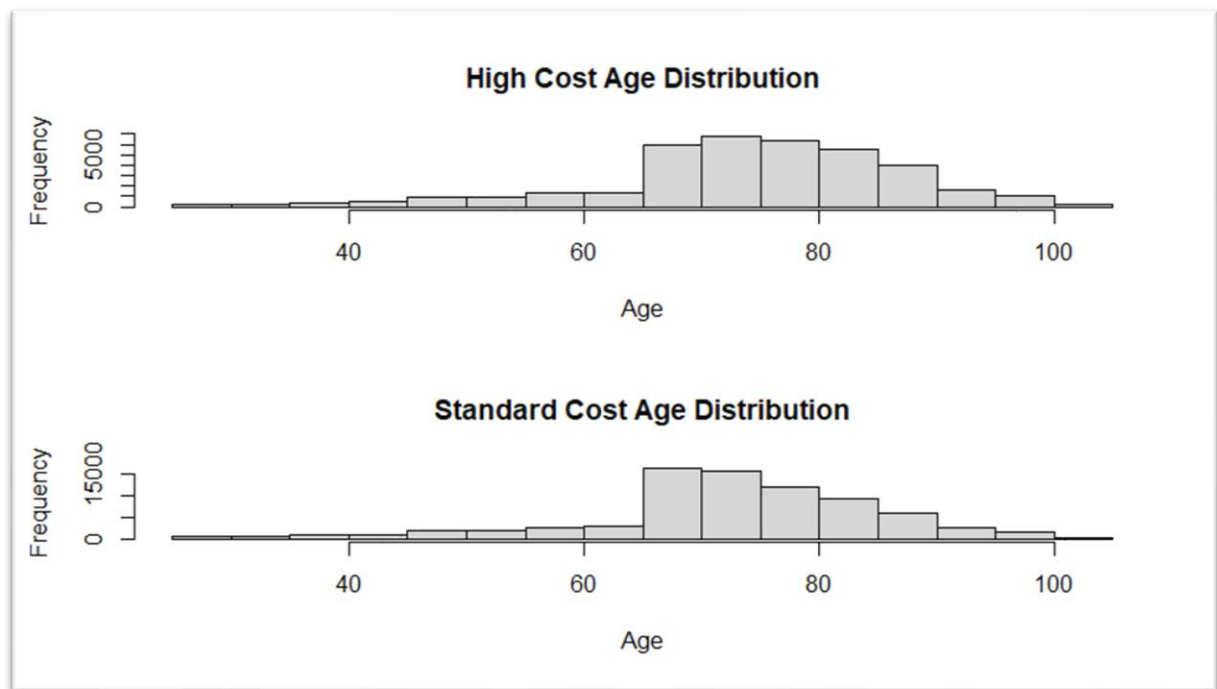
EDA Visualization



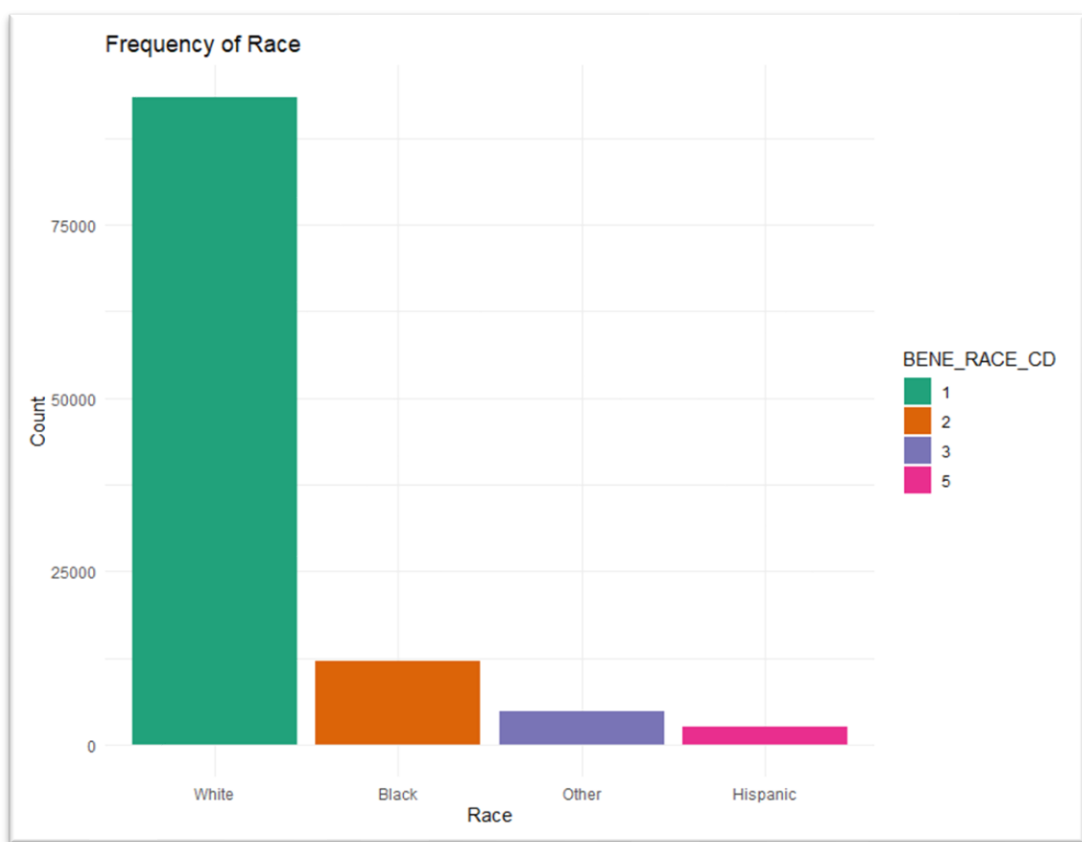
EDA Chart1: Medicare Cost Transformation



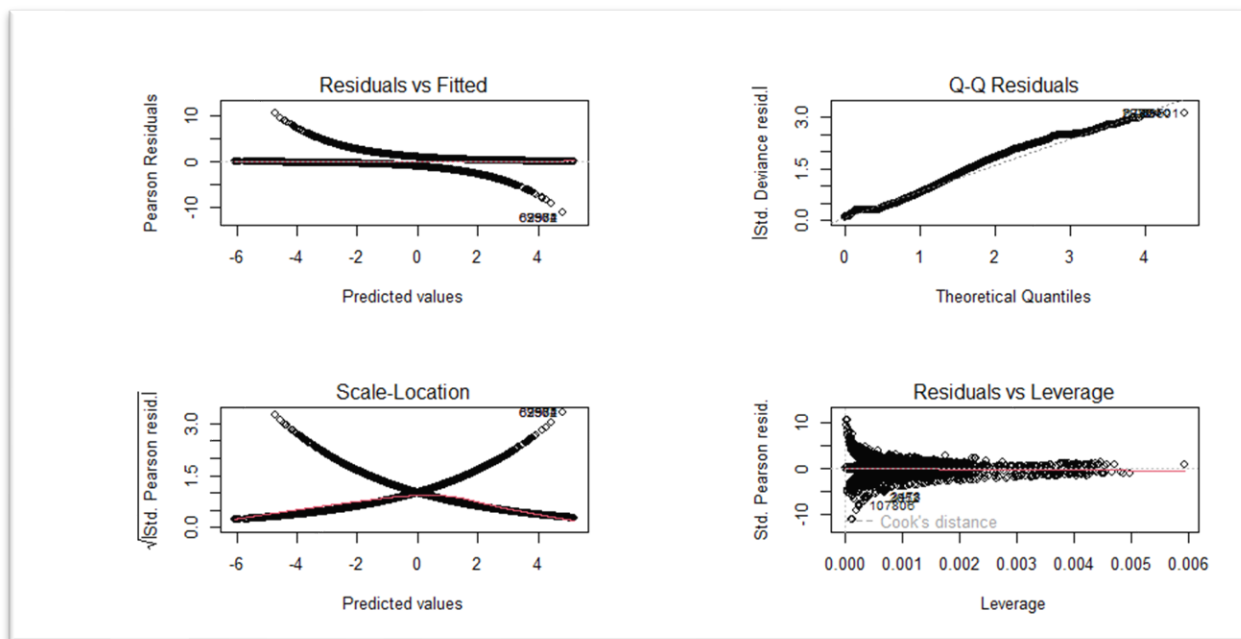
EDA Chart2: Chronic Disease Distribution



EDA Chart 3: Age Distribution by Cost Factor

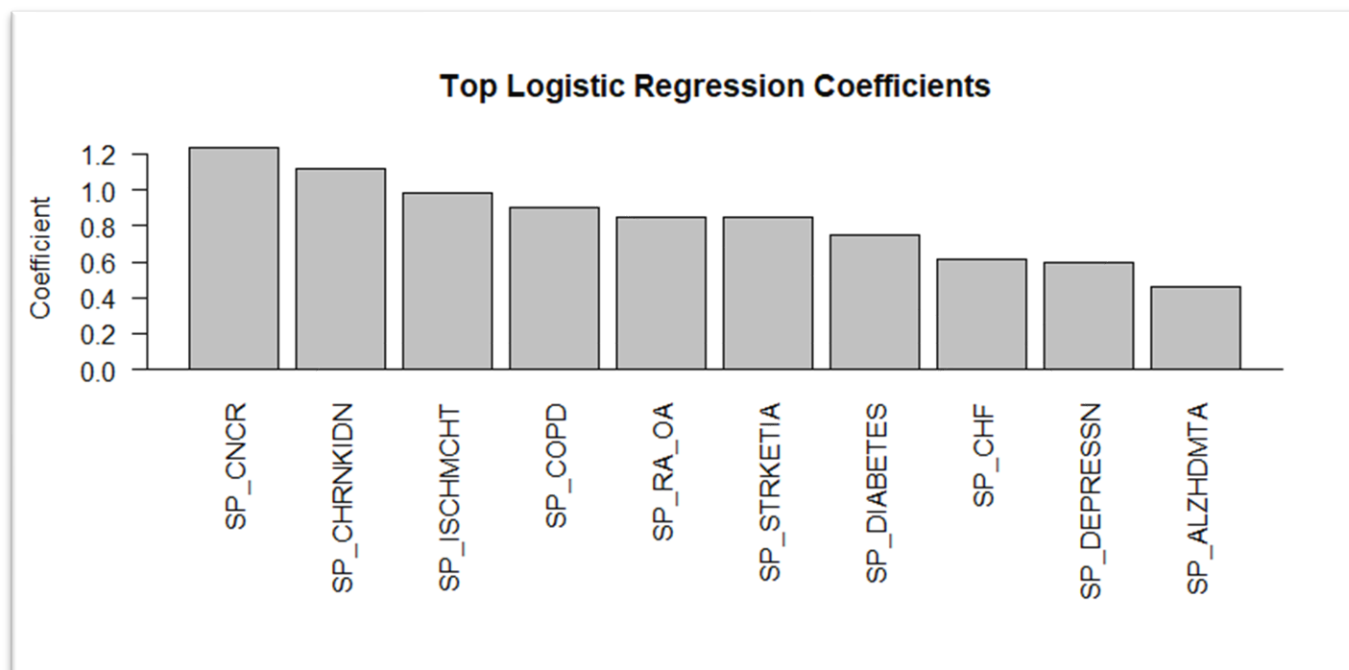


EDA Chart 4: Distribution by Race

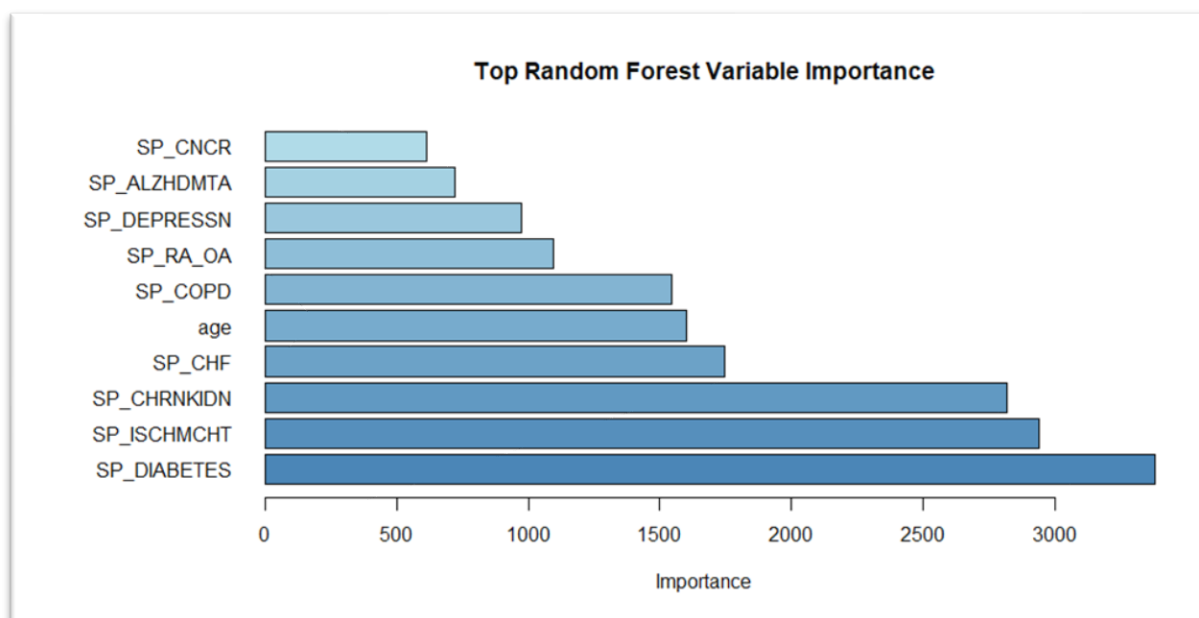
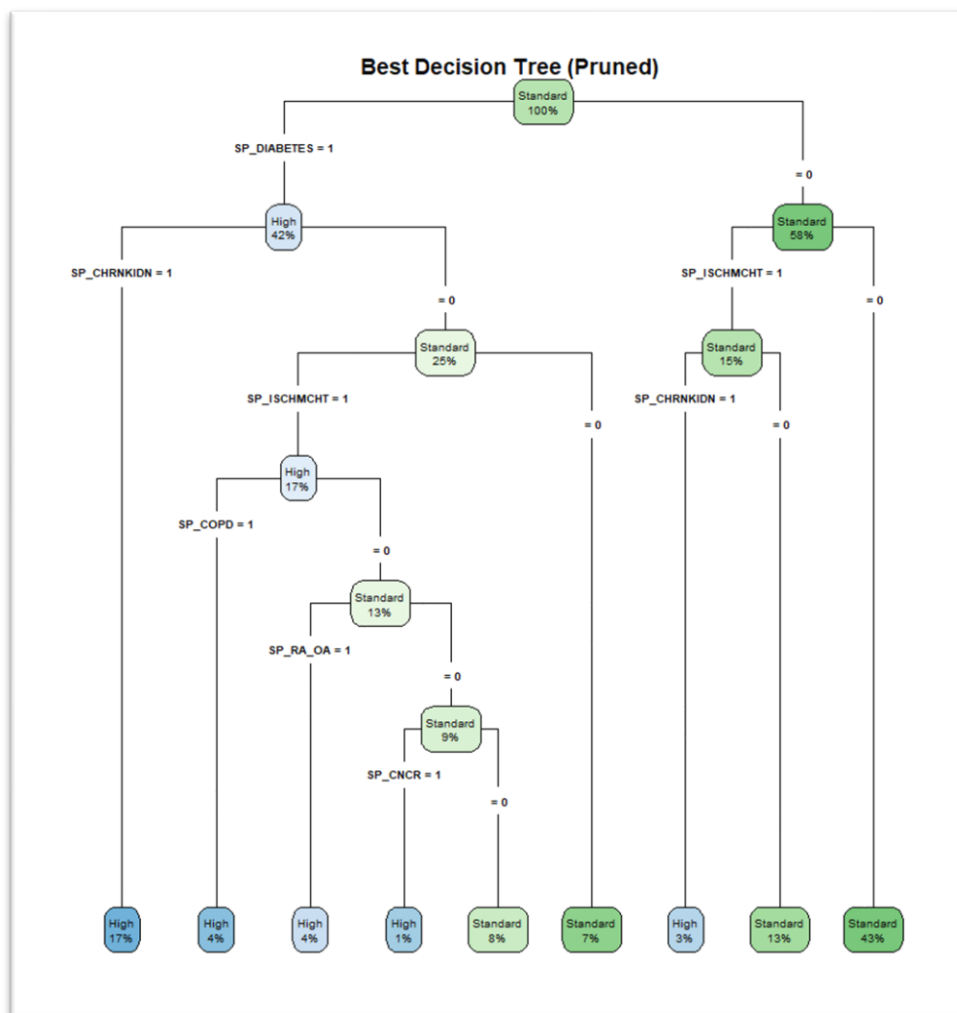


EDA Chart 5: Logistic regression Assumption Check

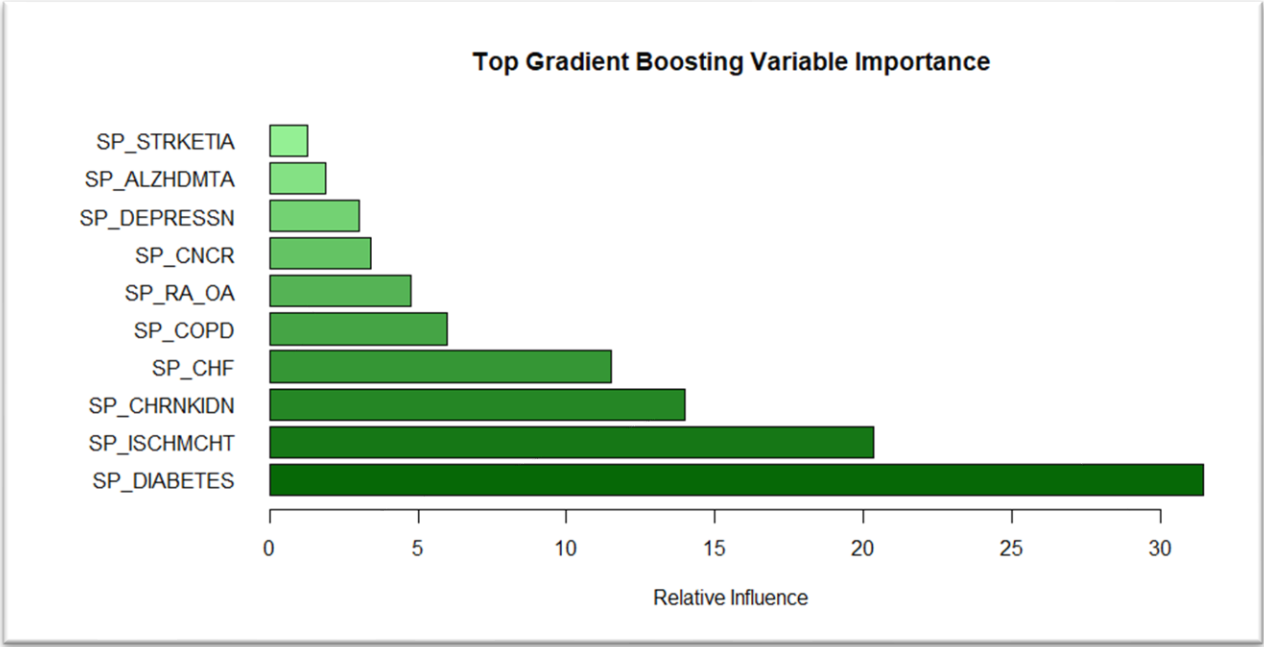
Variable Influence Visualizations



Variable Influence Chart1: Logistic regression



Variable Influence Chart 2: Decision Tree



Variable Influence Chart 3: Random Forest

Variable Influence Chart 4: Gradient Boosting

Model Hyper Parameter Tuning Results

Logistic regression Hyperparameter Tuning			
(≤ Threshold)	Recall	F1	Kappa
0.5	0.672	0.716	0.590
0.6	0.756	0.738	0.605
0.7	0.826	0.739	0.589
0.8	0.897	0.727	0.552

Table 1: Logistic regression Tuning Results

Decision Tree Hyperparameter Tuning					
cp	minsplit	minbucket	Recall	F1	Kappa
0.0005	500	167	0.699	0.720	0.589
0.0010	750	250	0.698	0.718	0.585
0.0010	500	167	0.695	0.718	0.586
0.0005	750	250	0.690	0.718	0.587
0.0001	750	250	0.678	0.711	0.579
0.0001	500	167	0.676	0.711	0.579
0.0050	500	167	0.647	0.697	0.565
0.0050	750	250	0.647	0.697	0.565

Table 2: Decision Tree Tuning Results

Random Forest Hyperparameter Tuning					
mtry	min.node.size	num.trees	Recall	F1	Kappa
4	12	500	0.690	0.721	0.593
5	12	500	0.689	0.718	0.589
5	6	500	0.689	0.718	0.587
4	6	500	0.688	0.720	0.592
5	2	500	0.686	0.717	0.586
4	2	500	0.686	0.719	0.590

Table 3: Random Forest Tuning Results

Gradient Boosting Hyperparameter Tuning							
n.trees	interaction. depth	shrinkage	n. minobsinnode	best_iter	Recall	F1	kappa
300	5	0.05	30	242	0.702	0.726	0.598
300	5	0.1	10	141	0.702	0.726	0.598
300	5	0.1	30	132	0.700	0.726	0.598
300	5	0.05	10	300	0.700	0.726	0.598
300	5	0.1	20	152	0.699	0.725	0.597
300	5	0.05	20	278	0.699	0.725	0.597

Table 4: Gradient Boosting Tuning Results

K Nearest Neighbor Hyperparameter Tuning			
k	Recall	F1	kappa
21	0.685	0.711	0.577
7	0.681	0.697	0.553
31	0.681	0.712	0.581
15	0.681	0.706	0.569
11	0.680	0.703	0.564
3	0.672	0.680	0.526

Table 5: K Nearest Neighbor Tuning Results

Final Results

Best Model Results			
Model	Recall	F1	kappa
Logistic_Classification	0.829	0.737	0.585
Boost_Model	0.708	0.727	0.598
Random_Forrest	0.696	0.722	0.593
Decision_Tree	0.693	0.711	0.574
KNN_Model	0.687	0.712	0.577

Table 6: Best Model Results

McNemar Model Comparison to Logistic regression		
Model_Compared	Chi_Square	P_Value
Gradient Boosting	1667	0.000
Random Forest	429	0.000
Decision Tree	616	0.000
K Nearest Neighbor	1688	0.000

Variable Definition Appendix

Variable #	Variable Name	Label: Description
1	DESYNPUF_ID	DESYNPUF: Beneficiary Code
2	BENE_BIRTH_DT	DESYNPUF: Date of birth
3	BENE_DEATH_DT	DESYNPUF: Date of death
4	BENE_SEX_IDENT_CD	DESYNPUF: Sex
5	BENE_RACE_CD	DESYNPUF: Beneficiary Race Code
6	BENE_ESRD_IND	DESYNPUF: End stage renal disease Indicator
7	SP_STATE_CODE	DESYNPUF: State Code
8	BENE_COUNTY_CD	DESYNPUF: County Code
9	BENE_HI_CVRAGE_TOT_MONS	DESYNPUF: Total number of months of part A coverage for the beneficiary
10	BENE_SMI_CVRAGE_TOT_MONS	DESYNPUF: Total number of months of part B coverage for the beneficiary
11	BENE_HMO_CVRAGE_TOT_MONS	DESYNPUF: Total number of months of HMO coverage for the beneficiary
12	PLAN_CVRG_MOS_NUM	DESYNPUF: Total number of months of part D plan coverage for the beneficiary
13	SP_ALZHDMTA	DESYNPUF: Chronic Condition: Alzheimer or related disorders or senile
14	SP_CHF	DESYNPUF: Chronic Condition: Heart Failure
15	SP_CHRNKIDN	DESYNPUF: Chronic Condition: Chronic Kidney Disease
16	SP_CNCR	DESYNPUF: Chronic Condition: Cancer
17	SP_COPD	DESYNPUF: Chronic Condition: Chronic Obstructive Pulmonary Disease
18	SP_DEPRESSN	DESYNPUF: Chronic Condition: Depression
19	SP_DIABETES	DESYNPUF: Chronic Condition: Diabetes
20	SP_ISCHMCHT	DESYNPUF: Chronic Condition: Ischemic Heart Disease
21	SP_OSTEOPRS	DESYNPUF: Chronic Condition: Osteoporosis
22	SP_RA_OA	DESYNPUF: Chronic Condition: Rheumatoid arthritis and osteoarthritis (RA/OA)
23	SP_STRKETIA	DESYNPUF: Chronic Condition: Stroke/transient Ischemic Attack
24	MEDREIMB_IP	DESYNPUF: Inpatient annual Medicare reimbursement amount
25	BENRES_IP	DESYNPUF: Inpatient annual beneficiary responsibility amount
26	PPPYMT_IP	DESYNPUF: Inpatient annual primary payer reimbursement amount
27	MEDREIMB_OP	DESYNPUF: Outpatient Institutional annual Medicare reimbursement amount
28	BENRES_OP	DESYNPUF: Outpatient Institutional annual beneficiary responsibility amount
29	PPPYMT_OP	DESYNPUF: Outpatient Institutional annual primary payer reimbursement amount
30	MEDREIMB_CAR	DESYNPUF: Carrier annual Medicare reimbursement amount
31	BENRES_CAR	DESYNPUF: Carrier annual beneficiary responsibility amount
32	PPPYMT_CAR	DESYNPUF: Carrier annual primary payer reimbursement amount

Code Appendix

```
library(dplyr)
library(caret)
library(rpart)
library(rpart.plot)
library(randomForest)
library(ranger)
library(gbm)
library(xgboost)
library(ggplot2)
library(car)

data = read.csv('DE1_0_2009_Beneficiary_Summary_File_Sample_1.csv', header = TRUE)
dim(data)
summary(data)
colnames(df)
df = data

df = df[is.na(df$BENE_DEATH_DT), ]
df[4:6] = lapply(df[4:6], as.factor)
df[13:23] = lapply(df[13:23], as.factor)
df$BENE_BIRTH_DT = as.Date(as.character(df$BENE_BIRTH_DT), format = "%Y%m%d")
df$age = round(as.numeric(difftime(as.Date("2009-12-31"), df$BENE_BIRTH_DT, units = "days")) / 365,1)

plot(df$BENE_BIRTH_DT)
hist(df$age)
northeast = c("09", "23", "25", "33", "44", "50", "34", "36", "42")
south = c("10", "11", "12", "13", "24", "37", "45", "51", "54", "01", "21", "28", "47", "05", "22", "40", "48")
midwest = c("17", "18", "26", "39", "55", "19", "20", "27", "29", "31", "38", "46")
west = c("04", "08", "16", "35", "30", "32", "49", "56", "02", "06", "15", "41", "53")
other = c("72", "78", "54")

df = df %>%
  mutate(region = case_when(
    SP_STATE_CODE %in% northeast ~ "Northeast",
    SP_STATE_CODE %in% south ~ "South",
    SP_STATE_CODE %in% midwest ~ "Midwest",
    SP_STATE_CODE %in% west ~ "West",
    TRUE ~ "Other"
  ))

chronic_vars = c("SP_ALZHDMTA", "SP_CHF", "SP_CHRNKIDN", "SP_CNCR",
  "SP_COPD", "SP_DEPRESSN", "SP_DIABETES", "SP_ISCHMCHT",
  "SP_OSTEOPRS", "SP_RA_OA", "SP_STRKETIA")

df = df %>%
  mutate(across(all_of(chronic_vars), ~ ifelse(. == 1, 1, 0)))

df$BENE_SEX_IDENT_CD = ifelse(df$BENE_SEX_IDENT_CD == 1, 1, 0)

df$BENE_ESRD_IND = ifelse(df$BENE_ESRD_IND == "Y", 1, 0)
df$BENE_ESRD_IND = as.integer(df$BENE_ESRD_IND)

df$MEDERE_COST = df$MEDREIMB_IP + df$MEDREIMB_OP + df$MEDREIMB_CAR
df = df %>% filter(df$MEDERE_COST >= 0)

df$MEDERE_COST_Log = log(df$MEDERE_COST + 1)
standard = quantile(df$MEDERE_COST_Log, probs = c(0.67))

df$Cost_Factor = ifelse(df$MEDERE_COST_Log <= standard,
  "Standard", "High")

df = df %>%
  select(-SP_STATE_CODE, -BENE_COUNTY_CD, -DESYNPUF_ID, -BENE_DEATH_DT,
    -BENRES_IP, -BENRES_OP, -BENRES_CAR, -PPPYMT_CAR, -PPPYMT_OP,
    -PPPYMT_IP, -BENE_BIRTH_DT)

#EDA/GRAPHS
par(mfrow = c(2, 1))
hist(df$MEDERE_COST,
  main = "Raw Medicare Cost",
  xlab = "Cost ($)",
  col = "steelblue",
  breaks = 50)
hist(df$MEDERE_COST_Log,
  main = "Log Transformed Cost",
  xlab = "Log Cost",
```

```

col = "steelblue",
breaks = 50)

par(mfrow = c(3, 4))

for (var in chronic_vars) {
  rate = tapply(df[[var]], df$Cost_Factor, mean)
  barplot(rate,
    main = var,
    col = c("steelblue", "tomato"),
    ylab = "Proportion",
    ylim = c(0, 1))
}

par(mfrow = c(2, 1))

hist(df$age[df$Cost_Factor == "High"],
  main = "High Cost Age Distribution",
  xlab = "Age")

hist(df$age[df$Cost_Factor == "Standard"],
  main = "Standard Cost Age Distribution",
  xlab = "Age")

prop.table(table(df$BENE_RACE_CD))

ggplot(df, aes(x = factor(BENE_RACE_CD,
  levels = c(1, 2, 3, 5),
  labels = c("White", "Black", "Other",
    "Hispanic"))
, fill = BENE_RACE_CD)) +
  geom_bar() +
  scale_fill_brewer(palette = "Dark2") +
  labs(title = "Frequency of Race",
    x = "Race",
    y = "Count") +
  theme_minimal()

df_model = df %>% select ( -MEDERE_COST_Log, -MEDERE_COST,
  -MEDREIMB_IP, -MEDREIMB_OP, -MEDREIMB_CAR) %>%
  mutate(across(c(Cost_Factor, region), as.factor))

####Data Split###
set.seed(123)
train_index = createDataPartition(df_model$Cost_Factor, p = 0.7, list = FALSE)
train = df_model[train_index,]
step = df_model[-train_index,]
val_index = createDataPartition(step$Cost_Factor, p = 0.5, list = FALSE)
validation = step[val_index,]
test = step[-val_index,]
round(prop.table(table(df$Cost_Factor)),3)
round(prop.table(table(train$Cost_Factor)),3)
round(prop.table(table(validation$Cost_Factor)),3)
round(prop.table(table(test$Cost_Factor)),3)
#####Logistic Baseline#####

Logistic_model = glm(Cost_Factor ~., data = train, family = binomial)
vif = vif(Logistic_model)
par(mfrow = c(2,2))
plot(Logistic_model)
par(mfrow = c(2,1))

##### Tree
tree_model = rpart(Cost_Factor ~., data = train, method = "class")

##### Forest
rf_model = ranger(Cost_Factor ~., data = train,
  num.trees = 500,
  min.node.size = 20,
  importance = "impurity")

##### Boost
train_num = train
train_num$Cost_Factor = ifelse(train$Cost_Factor == 'High', 1,0)
boost_model = gbm(Cost_Factor ~.,
  data = train_num,

```

```

distribution = "bernoulli",
n.trees = 100,
interaction.depth = 3,
shrinkage = 0.3,
n.minobsinnode = 10,
verbose = TRUE)

#####KNN PreProcess #####
factor_cols = setdiff(names(train), c("age", "BENE_HI_CVRAGE_TOT_MONS", "BENE_SMI_CVRAGE_TOT_MONS",
"BENE_HMO_CVRAGE_TOT_MONS", "PLAN_CVRG_MOS_NUM"))

PP_train = train
PP_validation = validation
PP_Test = test

PP_train[factor_cols] = lapply(PP_train[factor_cols], as.factor)
PP_validation[factor_cols] = lapply(PP_validation[factor_cols], as.factor)
PP_Test[factor_cols] = lapply(PP_Test[factor_cols], as.factor)

y_train = PP_train$Cost_Factor
y_valid = PP_validation$Cost_Factor
y_test = PP_Test$Cost_Factor

x_train = PP_train %>% select(-Cost_Factor)
x_valid = PP_validation %>% select(-Cost_Factor)
x_test = PP_Test %>% select(-Cost_Factor)

dummies = dummyVars(~., data = x_train)
x_train_dummy = predict(dummies, newdata = x_train) %>% as.data.frame()
x_valid_dummy = predict(dummies, newdata = x_valid) %>% as.data.frame()
x_test_dummy = predict(dummies, newdata = x_test) %>% as.data.frame()

pp = preProcess(x_train_dummy, method = c("center", "scale"))

x_train_scaled = predict(pp, x_train_dummy)
x_valid_scaled = predict(pp, x_valid_dummy)
x_test_scaled = predict(pp, x_test_dummy)

#####
###VALIDATION/Parameter Tuning#####
#####

####Logistics
Logistic_Results = data.frame()

log_pred = predict(Logistic_model, newdata = validation, type = "response")
threshold = c(.5, .6, .7, .8)

for (t in threshold) {
  log_class = ifelse(log_pred <= t, "High", "Standard")
  conf <- confusionMatrix(as.factor(log_class), validation$Cost_Factor, positive = "High")

  Logistic_Results <- rbind(Logistic_Results,
    data.frame(
      threshold = t,
      Recall = conf$byClass["Sensitivity"],
      F1 = conf$byClass["F1"],
      kappa = conf$overall["Kappa"]
    ))
}

Logistic_Results
##### Tree
Tree_results = data.frame()

tree_params = expand.grid(
  cp = c(0.0001, .0005, .001, .005),
  minsplit = c(500, 750)
)

for (i in 1:nrow(tree_params)) {

  tree_model = rpart(
    Cost_Factor ~.,
    data = train,
    method = "class",
    control = rpart.control(
      cp = tree_params$cp[i],
      minsplit = tree_params$minsplit[i]
    )
  )

```

```

)

tree_pred = predict(tree_model, validation, type = "class")

conf = confusionMatrix(tree_pred, validation$Cost_Factor, positive = "High")

Recall = conf$byClass["Sensitivity"]
F1 = conf$byClass["F1"]

kappa = conf$overall["Kappa"]

Tree_results = rbind(Tree_results,
  cbind(
    tree_params[i, ],
    minbucket = tree_model$control$minbucket,
    Recall = Recall,
    f1_Score = F1,
    Cohen_Kappa_Measure = kappa
  ))
}

Tree_results
#####Forest

RF_results = data.frame()

RF_params = expand.grid(
  mtry = c(4, 5),
  min.node.size = c(2, 6, 12)
)

for (i in 1:nrow(RF_params)) {

  rf_model = ranger(
    Cost_Factor ~ .,
    data = train,
    mtry = RF_params$mtry[i],
    min.node.size = RF_params$min.node.size[i],
    splitrule = "gini",
    num.trees = 500,
    importance = "impurity",
    probability = FALSE
  )

  rf_pred = predict(rf_model, data = validation)$predictions

  conf = confusionMatrix(rf_pred, validation$Cost_Factor)

  Recall = conf$byClass["Sensitivity"]

  F1 = conf$byClass["F1"]

  kappa = conf$overall["Kappa"]

  RF_results = rbind(
    RF_results,
    data.frame(
      mtry = RF_params$mtry[i],
      min.node.size = RF_params$min.node.size[i],
      num.trees = 500,
      Recall = Recall,
      f1_Score = F1,
      Cohen_Kappa_Measure = kappa
    )
  )
}
RF_results

##### Boost
validation_num = validation
validation_num$Cost_Factor = ifelse(validation$Cost_Factor == 'High', 1, 0)

test_num = test
test_num$Cost_Factor = ifelse(test$Cost_Factor == 'High', 1, 0)

Boost_results = data.frame()
Boost_params = expand.grid(
  shrinkage = c(0.05, 0.1),

```

```

n.minobsinnode = c(10, 20, 30)
)

for (i in 1:nrow(Boost_params)) {

  boost_model = gbm(Cost_Factor ~.,
    data = train_num,
    distribution = "bernoulli",
    n.trees = 300,
    interaction.depth = 5,
    shrinkage = Boost_params$shrinkage[i],
    n.minobsinnode = Boost_params$n.minobsinnode[i],
    cv.folds = 5,
    verbose = TRUE)

  best_iter = gbm.perf(boost_model, method = "cv", plot.it = FALSE)

  boost_prob = predict(boost_model,
    newdata = validation_num,
    n.trees = best_iter,
    type = "response")

  boost_class = ifelse(boost_prob <= 0.5, "Standard", "High")
  boost_class = factor(boost_class, levels = levels(validation$Cost_Factor))

  conf = confusionMatrix(boost_class, validation$Cost_Factor)

  Recall = conf$byClass["Sensitivity"]

  F1 = conf$byClass["F1"]

  kappa = conf$overall["Kappa"]

  Boost_results = rbind(
    Boost_results,
    data.frame(
      n.trees = boost_model$n.trees,
      interaction.depth = 5,
      shrinkage = Boost_params$shrinkage[i],
      n.minobsinnode = Boost_params$n.minobsinnode[i],
      best_iter = best_iter,
      Recall = Recall,
      F1 = F1,
      kappa = kappa
    ))
  Boost_results
  #### KNN
  KNN_Results = data.frame()
  k_vals = c(3, 7, 11, 15, 21, 31)
  for (i in seq_along(k_vals)) {
    k = k_vals[i]
    pred = knn(
      train = x_train_scaled,
      test = x_valid_scaled,
      cl = y_train,
      k = k
    )
    conf = confusionMatrix(pred, y_valid, positive = "High")
    recall = conf$byClass["Sensitivity"]
    F1 = conf$byClass["F1"]
    kappa = conf$overall["Kappa"]
    KNN_Results = rbind(
      KNN_Results,
      data.frame(
        k = k,
        Recall = recall,
        F1 = F1,
        kappa = kappa
      ))
  }
  KNN_Results
  # Sort by selection criteria
  Logistic_Results = data.frame(Logistic_Results, row.names = NULL)
  Logistic_Results[] = lapply(Logistic_Results, as.numeric)
  Top_Log_Results = Logistic_Results[order(
    -Logistic_Results$F1,
    -Logistic_Results$Recall,
    -Logistic_Results$kappa),]
  Top_Log_Results1 = Top_Log_Results[1,]

```

```

Top_Log_Results1
Tree_results
Tree_results[] = lapply(Tree_results, as.numeric)
Top_val_tree = Tree_results[order(
  -Tree_results$Recall,
  -Tree_results$f1_Score ,
  -Tree_results$Cohen_Kappa_Measure),]
Top_val_tree1 = Top_val_tree[1,]
Top_val_tree1
RF_results = data.frame(RF_results, row.names = NULL)
RF_results[] = lapply(RF_results, as.numeric)
Top_val_rf = RF_results[order(
  -RF_results$Recall ,
  -RF_results$f1_Score ,
  -RF_results$Cohen_Kappa_Measure ), ]
Top_val_rf1 = Top_val_rf[1,]
Top_val_rf1
Boost_results = data.frame(Boost_results, row.names = NULL)
Boost_results[] = lapply(Boost_results, as.numeric)
Top_val_boost = Boost_results[order(
  -Boost_results$Recall,
  -Boost_results$F1 ,
  -Boost_results$kappa),]
Top_val_boost1 = Top_val_boost[1, ]
Top_val_boost1
KNN_Results = data.frame(KNN_Results, row.names = NULL)
KNN_Results[] = lapply(KNN_Results, as.numeric)
Top_val_KNN = KNN_Results[order(
  -KNN_Results$Recall,
  -KNN_Results$F1,
  -KNN_Results$kappa),]

Top_val_KNN1 = Top_val_KNN[1,]
##### TEST SET #####
###Retrain/test with best parameters###
log_pred = predict(Logistic_model, newdata = test, type = "response")
log_class = ifelse(log_pred <= Top_Log_Results1$threshold, "High", "Standard")
log_class = as.factor(log_class)
Log_conf = confusionMatrix(as.factor(log_class), test$Cost_Factor, positive = "High")
Log_conf
par(mfrow = c(2,1))
coef_df = data.frame(Variable = names(Logistic_model$coefficients[-1]),
  Coefficient = abs(as.numeric(Logistic_model$coefficients[-1])))
coef_df = coef_df[order(-(coef_df$Coefficient)), ][1:10,]
barplot(
  coef_df$Coefficient,
  names.arg = coef_df$Variable,
  las = 2,
  main = "Top Logistic Regression Coefficients",
  ylab = "Coefficient"
)
abline(h = 0)
Best_Tree = rpart(Cost_Factor~., data = train,
  cp = Top_val_tree1$cp,
  minsplit = Top_val_tree1$minsplit,
  minbucket = Top_val_tree1$minbucket)
Tree_Pred = predict(Best_Tree, test, type = 'class')
Tree_conf = confusionMatrix(Tree_Pred, test$Cost_Factor)
Best_Tree$variable.importance[1:10]
summary(tree_model)
par(mfrow = c(1,1))
pruned_tree <- prune(Best_Tree, cp = 0.005)
rpart.plot(pruned_tree, type = 4, extra = 100, fallen.leaves = TRUE, tweak = 1)
title("Best Decision Tree (Pruned)")
Best_RF = ranger(Cost_Factor~., data = train,
  mtry = Top_val_rf1$mtry,
  min.node.size = Top_val_rf1$min.node.size ,
  num.trees = Top_val_rf1$num.trees,
  importance = "impurity")
RF_Pred = predict(Best_RF, test, type = 'response')$prediction
RF_conf = confusionMatrix(RF_Pred, test$Cost_Factor)

par(mfrow = c(2,1))
var_imp <- sort(Best_RF$variable.importance, decreasing = TRUE)[1:10]
barplot(var_imp,
  horiz = 'TRUE',
  las = 1, # rotate labels
  col = colorRampPalette(c("steelblue", "lightblue"))(10),
  main = "Top Random Forest Variable Importance",
  xlab = "Importance")

```

```

Best_Boost = gbm(Cost_Factor~., data = train_num,
  distribution = "bernoulli",
  n.trees = Top_val_boost1$best_iter,
  interaction.depth = Top_val_boost1$interaction.depth ,
  shrinkage = Top_val_boost1$shrinkage,
  n.minobsinnode = Top_val_boost1$n.minobsinnode,
  verbose = TRUE)
Boost_Pred = predict(Best_Boost, test_num, type = 'response')
Boost_Class = ifelse(Boost_Pred <= .5, "Standard", "High")
Boost_Class = as.factor(Boost_Class)
Boost_conf = confusionMatrix(Boost_Class, test$Cost_Factor)
par(mfrow = c(1,1))
boost_summary <- summary(Best_Boost, plotit = FALSE)
top10 <- boost_summary[1:10, ]
par(mar = c(5, 10, 4, 2)) # more space on left
barplot(top10$rel.inf,
  names.arg = top10$var,
  horiz = TRUE,
  las = 1,
  col = colorRampPalette(c("darkgreen","lightgreen"))(10),
  main = "Top Gradient Boosting Variable Importance",
  xlab = "Relative Influence")
Best_KNN = knn(
  train = x_train_scaled,
  test = x_test_scaled,
  cl = y_train,
  k = Top_val_KNN$k[1]
)
Knn_conf = confusionMatrix(Best_KNN, y_test, positive = "High")
c_results = list(Logistic_Classification = Log_conf,
  Decision_Tree = Tree_conf,
  Random_Forest = RF_conf,
  Boost_Model = Boost_conf,
  KNN_Model = Knn_conf)
Model_results = data.frame()
Model_results = data.frame(
  Model = names(c_results),
  Recall = sapply(c_results, function(x) x$byClass["Recall"]),
  F1 = sapply(c_results, function(x) x$byClass["F1"]),
  kappa = sapply(c_results, function(x) x$overall["Kappa"]),
  row.names = NULL)
Model_results = Model_results %>%
  arrange(desc(Recall), desc(F1), desc(kappa))
Model_results
Mcnemar_Results = data.frame(Model_Compared = character(),
  Chi_Square = numeric(),
  P_Value = numeric())

tb1 <- table(log_class, Boost_Class )
Mcnemar_Results = Mcnemar_Results %>%
  add_row(Model_Compared = "Gradient Boosting",
  Chi_Square = as.numeric(mcnemar.test(tb1)$statistic),
  P_Value = round(mcnemar.test(tb1)$p.value,3))
tb2 <- table(log_class, rf_pred)
mcnemar.test(tb2)

Mcnemar_Results = Mcnemar_Results %>%
  add_row(Model_Compared = "Random Forest",
  Chi_Square = as.numeric(mcnemar.test(tb2)$statistic),
  P_Value = round(mcnemar.test(tb2)$p.value,3))
tb3 <- table(log_class, tree_pred)
mcnemar.test(tb3)
Mcnemar_Results = Mcnemar_Results %>%
  add_row(Model_Compared = "Decision Tree",
  Chi_Square = as.numeric(mcnemar.test(tb3)$statistic),
  P_Value = round(mcnemar.test(tb3)$p.value,3))
tb4 <- table(log_class, Best_KNN)
mcnemar.test(tb4)
Mcnemar_Results = Mcnemar_Results %>%
  add_row(Model_Compared = "K Nearest Neighbor",
  Chi_Square = as.numeric(mcnemar.test(tb4)$statistic),
  P_Value = round(mcnemar.test(tb4)$p.value,3 ))

Mcnemar_Results

```


Bibliography/Citations

Centers for Medicare and Medicaid Services. (2013, January 15). *CMS linkable 2008–2010 Medicare Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF): DE1.0 Sample 1 2009 Beneficiary Summary File* [Data set]. U.S. Department of Health and Human Services. <https://www.cms.gov/research-statistics-data-and-systems/downloadable-public-use-files/synpufs>

Centers for Medicare and Medicaid Services. (2013, January 15). *CMS linkable 2008–2010 Medicare Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF) user manual*. U.S. Department of Health and Human Services. https://www.cms.gov/research-statistics-data-and-systems/downloadable-public-use-files/synpufs/downloads/synpuf_dug.pdf

Centers for Medicare and Medicaid Services. (2013, February 20). *CMS linkable 2008–2010 Medicare Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF) codebook*. U.S. Department of Health and Human Services. <https://www.cms.gov/files/document/de-10-codebook.pdf-0>

GeeksforGeeks. (2024). *Gradient boosting in R*. <https://www.geeksforgeeks.org/deep-learning/gradient-boosting-in-r/>

GeeksforGeeks. (2024). *How to tune a decision tree in hyperparameter tuning*. <https://www.geeksforgeeks.org/machine-learning/how-to-tune-a-decision-tree-in-hyperparameter-tuning/>

GeeksforGeeks. (2024). *Hyperparameter tuning with R*. <https://www.geeksforgeeks.org/machine-learning/hyperparameter-tuning-with-r/>